



Instituto Infnet

2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Dicionários (Dictionaries)

Dicionários são coleções de pares chave-valor, não-ordenadas (antes do Python 3.7) ou ordenadas (a partir do Python 3.7) pela ordem de inserção, mutáveis e sem duplicatas nas chaves.

```
# Criação de Dicionários

# sintaxe : { chave : valor, ...}

pessoas = {'nome' : 'Gesiel Lopes', 'profissao' : 'Professor', 'idade': 46}
conceitos = {1 : 'DML', 2: 'DL', 3: 'D', 4: 'ND'}
variado = {'chave' : 123.26, 2 : 'valor', 'lista' : [1,2,3]}
vazio = {} # ou atraves da funcao built-in dict()

print(pessoas)
print(conceitos)
print(variado)
print(vazio)
```

```
{'nome': 'Gesiel Lopes', 'profissao': 'Professor', 'idade': 46}
{1: 'DML', 2: 'DL', 3: 'D', 4: 'ND'}
{'chave': 123.26, 2: 'valor', 'lista': [1, 2, 3]}
{}
```

```
# Criando dicionarios com a funcao built-in dict

pessoas_2 = dict(nome='Gesiel Lopes', profissao='Professor', idade=46)
print(pessoas_2)
```

```
{'nome': 'Gesiel Lopes', 'profissao': 'Professor', 'idade': 46}
```

```
nome = input('Digite seu nome:')
profissao = input('Digite sua profissao')

pessoas_3 = dict(nome, profissao)
print(pessoas_3)
```

Execution error

TypeError: dict expected at most 1 argument, got 2

```
-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 4
      1 nome = input('Digite seu nome:')
      2 profissao = input('Digite sua profissao')
----> 4 pessoas_3 = dict(nome, profissao)
      5 print(pessoas_3)
```

TypeError: dict expected at most 1 argument, got 2

Hide error details

```
# Acessando e Modificando
pessoas = {'nome' : 'Gesiel Lopes', 'profissao' : 'Professor', 'idade': 46}

# acessando os valores
nome = pessoas['nome']
print(nome)
```

Gesiel Lopes

```
# e se achave nao existir no dicionario?
```

```
passa_tempo = pessoas['passa_tempo']
print(passa_tempo)
```

❗ Execution error

KeyError: 'passa_tempo'

```
-----
KeyError                                Traceback (most recent call last)
Cell In[29], line 3
      1 # e se achave nao existir no dicionario?
----> 3 passa_tempo = pessoas['passa_tempo']
      4 print(passa_tempo)

KeyError: 'passa_tempo'
```

Hide error details

```
# modificar um valor em um dicionario
pessoas['nome'] = 'John Doe'
print(pessoas)

# acrescentar outras chaves e valores ao dicionario jah criado
pessoas['passa_tempo'] = 'Jogar com retro games'
print(pessoas)

passa_tempo = pessoas['passa_tempo']
print(passa_tempo)
```

```
{'nome': 'John Doe', 'profissao': 'Professor', 'idade': 46}
{'nome': 'John Doe', 'profissao': 'Professor', 'idade': 46, 'passa_tempo': 'Jogar com retro games'}
Jogar com retro games
```

```
teste_com_dicionario = dict()
for _ in range(3):
    chave = input('Digite uma chave')
    valor = int(input('Digite um numero'))

    teste_com_dicionario[chave] = valor

print(teste_com_dicionario)
```

```
{'chave1': 21, 'chave2': 22, 'chave3': 33}
```

```

# Métodos de Dicionários uteis
pessoas = {'nome' : 'Gesiel Lopes', 'profissao' : 'Professor', 'idade': 46}

# obter todas as chaves de um dicionario
chaves = pessoas.keys()

print(chaves, type(chaves))

chaves = list(chaves)
print(chaves, type(chaves))

print(80*'-')

# obter todos os valores
valores = pessoas.values()
print(valores, type(valores))

valores = list(valores)
print(valores, type(valores))

print(80*'-')

# obter todos os pares chave-valor
itens = pessoas.items()
print(itens, type(itens))

itens = list(itens)
print(itens, type(itens))

```

```

dict_keys(['nome', 'profissao', 'idade']) <class 'dict_keys'>
['nome', 'profissao', 'idade'] <class 'list'>
-----
dict_values(['Gesiel Lopes', 'Professor', 46]) <class 'dict_values'>
['Gesiel Lopes', 'Professor', 46] <class 'list'>
-----
dict_items([('nome', 'Gesiel Lopes'), ('profissao', 'Professor'), ('idade', 46)]) <class 'dict_items'>
[('nome', 'Gesiel Lopes'), ('profissao', 'Professor'), ('idade', 46)] <class 'list'>

```

```

# atualizando o dicionario com outro dicionario

pessoas = {'nome' : 'Gesiel Lopes', 'profissao' : 'Professor', 'idade': 46}
pessoas.update({'email' : 'gesiel.lope@prof.infnet.edu.br', 'linkedin' : '@gesielrios'})
print(pessoas)

pessoas.update({'nome' : 'John Doe', 'passa_tempo' : 'xadrez'})
print(pessoas)

```

```

{'nome': 'Gesiel Lopes', 'profissao': 'Professor', 'idade': 46, 'email': 'gesiel.lope@prof.infnet.edu.br', 'linkedin': '@gesielr
{'nome': 'John Doe', 'profissao': 'Professor', 'idade': 46, 'email': 'gesiel.lope@prof.infnet.edu.br', 'linkedin': '@gesielrios'

```

```

# obter e remover um item, com valor padrao se a chave nao existir
hobbie = pessoas.pop('hobbie', 'Nao informado')
print(hobbie)
print(pessoas)

email = pessoas.pop('email', 'Nao informado')
print(email)
print(pessoas)

```

```

Nao informado
{'nome': 'John Doe', 'profissao': 'Professor', 'idade': 46, 'email': 'gesiel.lope@prof.infnet.edu.br', 'linkedin': '@gesielrios'
gesiel.lope@prof.infnet.edu.br
{'nome': 'John Doe', 'profissao': 'Professor', 'idade': 46, 'linkedin': '@gesielrios', 'passa_tempo': 'xadrez'}

```

```

# Iterando sobre Dicionários
pessoas = {'nome' : 'Gesiel Lopes', 'profissao' : 'Professor', 'idade': 46}

print(50*'-')
# iterando sobre as chaves (comportado padrao)
for chave in pessoas.keys():
    print(chave, pessoas[chave])
print(50*'-')

for chave in pessoas:
    print(chave, pessoas[chave])
print(50*'-')

# iterando sobre os valores
for valor in pessoas.values():
    print(valor)
print(50*'-')

# iterando sobre pares chave valor
for retorno in pessoas.items():
    print(retorno, type(retorno))
print(50*'-')
for chave, valor in pessoas.items():
    print(f'{chave}: {valor}')

print(50*'-')

```

```

-----
nome Gesiel Lopes
profissao Professor
idade 46

```

```

-----
nome Gesiel Lopes
profissao Professor
idade 46

```

```

-----
Gesiel Lopes
Professor
46

```

```

-----
('nome', 'Gesiel Lopes') <class 'tuple'>
('profissao', 'Professor') <class 'tuple'>
('idade', 46) <class 'tuple'>

```

```

-----
nome: Gesiel Lopes
profissao: Professor
idade: 46

```

```

# A contagem dos tokens presentes no processamento do texto.

```

```

stopwords_br = [
    # Artigos e contrações
    'a', 'o', 'as', 'os', 'um', 'uma', 'uns', 'umas', 'ao', 'à', 'aos', 'às',
    'do', 'da', 'dos', 'das', 'no', 'na', 'nos', 'nas', 'pelo', 'pela', 'pelos', 'pelas',

    # Pronomes
    'eu', 'tu', 'ele', 'ela', 'nós', 'vós', 'eles', 'elas', 'me', 'mim', 'comigo',
    'te', 'ti', 'contigo', 'se', 'si', 'consigo', 'lhe', 'nos', 'vos', 'lhes',
    'meu', 'minha', 'meus', 'minhas', 'teu', 'tua', 'teus', 'tuas', 'seu', 'sua',
    'seus', 'suas', 'nosso', 'nossa', 'nossos', 'nossas', 'vosso', 'vossa', 'vossos', 'vossas',

    # Preposições
    'de', 'em', 'por', 'para', 'com', 'sem', 'sob', 'sobre', 'entre', 'até',
    'desde', 'contra', 'perante', 'através', 'além', 'dentro', 'fora', 'perto',
    'longe', 'durante',

    # Conjunções
    'e', 'mas', 'ou', 'porque', 'pois', 'que', 'se', 'como', 'quando', 'embora',
    'porém', 'todavia', 'contudo', 'então', 'também', 'apesar', 'caso', 'além disso',
    'portanto', 'logo', 'ainda que', 'a fim de', 'com o objetivo de',

    # Verbos auxiliares e comuns
    'ser', 'estar', 'ter', 'haver', 'ir', 'vir', 'fazer', 'poder', 'dever',
    'querer', 'saber', 'está', 'estão', 'tem', 'tenho', 'é',

    # Demonstrativos
    'esse', 'essa', 'isso', 'este', 'esta', 'isto', 'aquele', 'aquela', 'aquilo',
    'aqueles', 'aquelas', 'deste', 'desta', 'destes', 'destas', 'disso', 'daquilo',
    'nisto', 'naquilo',

    # Localizadores
    'lá', 'aqui', 'ali', 'onde', 'aonde'

    # Numericas
    'zero', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0'
]

def pre_processamento(stopwords, texto):
    """
        Definicao da funcao
        Etapas: Normalizacao, Tokenizacao (anagramas) e Remocao de Stopwords
    """
    texto = texto.lower() #Normalizacao
    tokens = texto.split() #Tokenizacao (anagramas)
    # list comprehension
    tokens = [token for token in tokens if token not in stopwords] # remocao das stopwords

    return tokens, ' '.join(tokens)

```

```

texto_ata_255 = '''Atualização da conjuntura econômica do cenário do Copom ambiente externo se mantém adverso Apesar da a
lista_tokens_preprocessado, texto_preprocessado = pre_processamento(stopwords=stopwords_br, texto=texto_ata_255)
lista_tokens_preprocessado[:10]

```

```

['atualização',
 'conjuntura',
 'econômica',
 'cenário',
 'copom',
 'ambiente',
 'externo',
 'mantém',
 'adverso',
 'atenuação']

```

```
contagem_tokens = dict()

for token in lista_tokens_preprocessado:

    if token in contagem_tokens.keys():
        contagem_tokens[token] += 1
    else:
        contagem_tokens[token] = 1

print(contagem_tokens)
```

```
{'atualização': 2, 'conjuntura': 5, 'econômica': 8, 'cenário': 9, 'copom': 12, 'ambiente': 2, 'externo': 1, 'mantém': 2, 'adverso
```

```
print(contagem_tokens['copom'])
```

```
12
```

```
# como obter o token com maior ocorrencia?
ocorrencia_token = 0
token_ocorrencia = dict()

for token, ocorrencia in contagem_tokens.items():

    if ocorrencia_token < ocorrencia:
        if len(token_ocorrencia.keys()) != 0:
            token_menor = list(token_ocorrencia.keys())[0]
            token_ocorrencia.pop(token_menor)

        ocorrencia_token = ocorrencia
        token_ocorrencia[token] = ocorrencia

print(token_ocorrencia)
```

```
{'inflação': 45}
```

Aprofundamento

- [Documentação sobre dicionários](#)